torch.compile#

https://pytorch.org/docs/stable/generated/torch.compile.html#torch.compile

Description:

Optimizes given model/function using TorchDynamo and specified backend. If you are compiling an torch.nn.Module, you can also use torch.nn.Module.compile() to compile the module inplace without changing its structure. Concretely, for every frame executed within the compiled region, we will attempt to compile it and cache the compiled result on the code object for future use. A single frame may be compiled multiple times if previous compiled results are not applicable for subsequent calls (this is called a “guard failure), you can use TORCH_LOGS=guards to debug these situations. Multiple compiled results can be associated with a frame up to torch._dynamo.config.recompile_limit, which defaults to 8; at which point we will fall back to eager. Note that compile caches are per code object, not frame; if you dynamically create multiple copies of a function, they will all share the same code cache. model (Callable or None) – Module/function to optimize fullgraph (bool) – If False (default), torch.compile attempts to discover compilable regions in the function that it will optimize. If True, then we require that the entire function be capturable into a single graph. If this is not pos

Function Signature

```
torch.compile(model: Callable[[_InputT], _RetT], *,
fullgraph: bool = False, dynamic: Optional[bool] = None,
backend: Union[str, Callable] = 'inductor', mode:
Optional[str] = None, options: Optional[dict[str, Union[str,
int, bool, Callable]]] = None, disable: bool = False) →
Callable[[_InputT], _RetT][source]#

torch.compile(model: None = None, *, fullgraph: bool =
False, dynamic: Optional[bool] = None, backend: Union[str,
Callable] = 'inductor', mode: Optional[str] = None, options:
Optional[dict[str, Union[str, int, bool, Callable]]] = None,
disable: bool = False) → Callable[[Callable[[_InputT],
_RetT]], Callable[[_InputT], _RetT]]

Parameters
```

Code Examples

```
@torch.compile(options={"triton.cudagraphs": True},
fullgraph=True)
def foo(x):
    return torch.sin(x) + torch.cos(x)
```

Detailed Documentation

Documentation

Optimizes given model/function using TorchDynamo and specified backend. If you are compiling an torch.nn.Module, you can also use torch.nn.Module.compile() to compile the module inplace without changing its structure.

Concretely, for every frame executed within the compiled region, we will attempt to compile it and cache the compiled result on the code object for future use. A single frame may be compiled multiple times if previous compiled results are not applicable for subsequent calls (this is called a “guard failure), you can use TORCH_LOGS=guards to debug these situations. Multiple compiled results can be associated with a frame up to torch._dynamo.config.recompile_limit, which defaults to 8; at which point we will fall back to eager. Note that compile caches are per code object, not frame; if you dynamically create multiple copies of a function, they will all share the same code cache.

model (Callable or None) – Module/function to optimize

fullgraph (bool) – If False (default), torch.compile attempts to discover compilable regions in the function that it will optimize. If True, then we require that the entire function be capturable into a single graph. If this is not possible (that is, if there are graph breaks), then this will raise an error.

dynamic (bool or None) – Use dynamic shape tracing. When this is True, we will up-front attempt to generate a kernel that is as dynamic as possible to avoid recompilations when sizes change. This may not always work as some operations/optimizations will force specialization; use TORCH_LOGS=dynamic to debug overspecialization. When this is False, we will NEVER generate dynamic kernels, we will always specialize. By default (None), we automatically detect if dynamism has occurred and compile a more dynamic kernel upon recompile.

backend (str or Callable) –

”inductor” is the default backend, which is a good balance between performance and overhead

Non experimental in-tree backends can be seen with torch._dynamo.list_backends()

Experimental or debug in-tree backends can be seen with torch._dynamo.list_backends(None)

To register an out-of-tree custom backend: https://pytorch.org/docs/main/torch.compiler_custom_backends.html#registering-custom-backends

Can be either “default”, “reduce-overhead”, “max-autotune” or “max-autotune-no-cudagraphs”

”default” is the default mode, which is a good balance between performance and overhead

”reduce-overhead” is a mode that reduces the overhead of python with CUDA graphs, useful for small batches. Reduction of overhead can come at the cost of more memory usage, as we will cache the workspace memory required for the invocation so that we do not have to reallocate it on subsequent runs. Reduction of overhead is not guaranteed to work; today, we only reduce overhead for CUDA only graphs which do not mutate inputs. There are other circumstances where CUDA graphs are not applicable; use TORCH_LOG=perf_hints to debug.

”max-autotune” is a mode that leverages Triton or template based matrix multiplications on supported devices and Triton based convolutions on GPU. It enables CUDA graphs by default on GPU.

”max-autotune-no-cudagraphs” is a mode similar to “max-autotune” but without CUDA graphs

To see the exact configs that each mode sets you can call torch._inductor.list_mode_options()

A dictionary of options to pass to the backend. Some notable ones to try out are

epilogue_fusion which fuses pointwise ops into templates. Requires max_autotune to also be set

max_autotune which will profile to pick the best matmul configuration

fallback_random which is useful when debugging accuracy issues

shape_padding which pads matrix shapes to better align loads on GPUs especially for tensor cores

triton.cudagraphs which will reduce the overhead of python with CUDA graphs

trace.enabled which is the most useful debugging flag to turn on

trace.graph_diagram which will show you a picture of your graph after fusion

guard_filter_fn that controls which dynamo guards are saved with compilations. This is an unsafe feature and there is no backward compatibility guarantee provided for dynamo guards as data types. For stable helper functions to use, see the documentations in torch.compiler, for example: - torch.compiler.skip_guard_on_inbuilt_nn_modules_unsafe - torch.compiler.skip_guard_on_all_nn_modules_unsafe - torch.compiler.keep_tensor_guards_unsafe

For inductor you can see the full list of configs that it supports by calling

torch._inductor.list_options()

disable (bool) – Turn torch.compile() into a no-op for testing

---